

CFGS: DESARROLLO DE APLICACIONES WEB

Curso: 2025-2026

GESTIÓN DE DOCENTES

Sistema integral de gestión docente para centros educativos de Formación Profesional

Gijón, mayo 2026

Autor/a: Vigil Díaz, Sergio

ÍNDICE

1. INTRODUCCIÓN

2. ESTUDIO DE MERCADO

3. ANÁLISIS Y DISEÑO DEL SISTEMA

3.1. Requisitos

3.2. Casos de uso

3.3. Diseño visual

3.4. Modelo de clases/datos

3.5. Diseño de la API

4. DESARROLLO TÉCNICO

4.1. Backend (Spring Boot)

4.2. Frontend

4.3. Herramientas de desarrollo

4.4. Despliegue

5. PLANIFICACIÓN Y PRESUPUESTO

5.1. Diagrama de Gantt

5.2. Presupuesto

6. FUTURAS MEJORAS Y AMPLIACIONES

7. VALORACIÓN Y CONCLUSIONES

8. ÍNDICE DE FIGURAS Y TABLAS

9. BIBLIOGRAFÍA/WEBGRAFÍA

ANEXOS

1. INTRODUCCIÓN

Descripción de la aplicación

GestiónDocentes es una aplicación web Full Stack desarrollada con arquitectura cliente-servidor que permite la gestión integral del profesorado en un centro de Formación Profesional. El sistema integra un backend construido con Spring Boot 3.3.2 que expone tanto vistas HTML renderizadas del lado del servidor mediante Thymeleaf como una API REST documentada con OpenAPI/Swagger, lo que posibilita la interoperabilidad con sistemas externos y la reutilización de datos por parte de aplicaciones de terceros.

La aplicación aborda una necesidad real en los centros educativos: la gestión centralizada de los docentes, sus horarios, las solicitudes de días de asuntos propios y la asignación automática de guardias cuando un docente se ausenta. Estos procesos, que en numerosos centros se gestionan todavía de forma manual o mediante hojas de cálculo, se automatizan en esta plataforma proporcionando trazabilidad, seguridad y eficiencia operativa.

El sistema implementa tres perfiles de usuario diferenciados (Dirección, Jefe de Departamento y Profesor), cada uno con contenidos, permisos y flujos de navegación adaptados a su rol. Además, la persistencia de datos se gestiona mediante JPA/Hibernate sobre una base de datos MySQL 8 normalizada, y la seguridad se delega en Spring Security con autenticación basada en formularios, cifrado BCrypt y control de acceso a nivel de método.

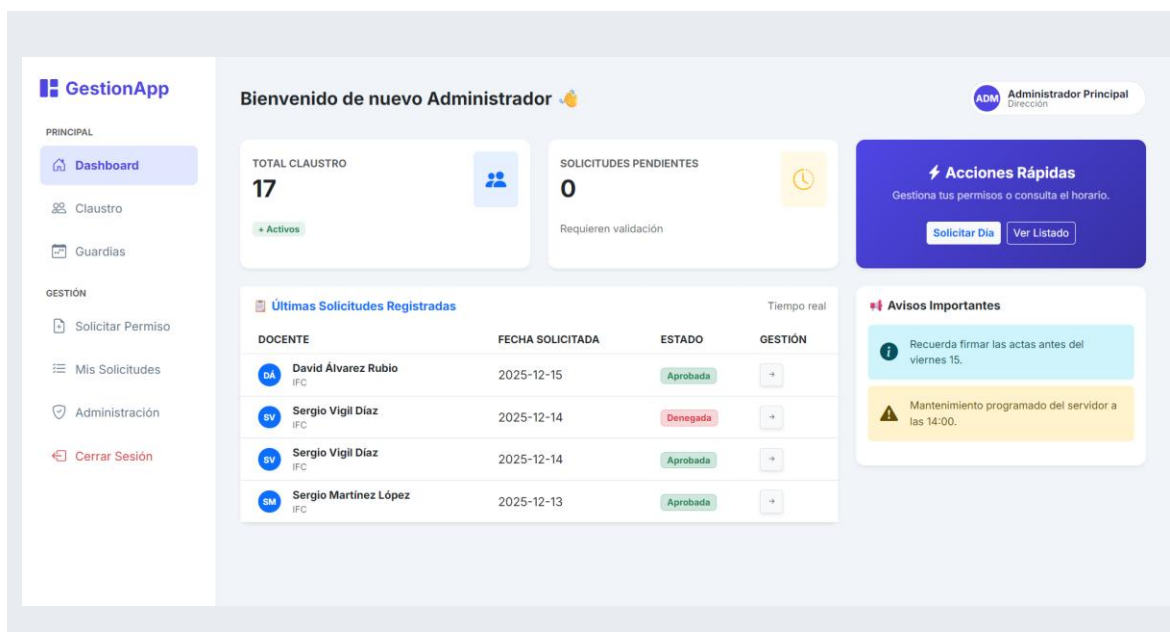


Figura 1. Vista general del dashboard de la aplicación (rol Dirección)

Objetivos del desarrollo

- Integrar los conocimientos adquiridos en los módulos de Desarrollo Web en Entorno Servidor (DWES), Desarrollo Web en Entorno Cliente (DWEC), Diseño de Interfaces Web (DIW) y Despliegue de Aplicaciones Web (DAW) en un único proyecto cohesivo.
- Diseñar e implementar una API REST que permita el consumo de servicios desde cualquier cliente HTTP, documentada de forma interactiva mediante Swagger UI.
- Aplicar patrones de diseño reconocidos como MVC (Model-View-Controller), inyección de dependencias y patrón repositorio para mantener un código desacoplado, testeable y mantenible.
- Garantizar la seguridad de la aplicación mediante autenticación robusta, autorización granular basada en roles y protección frente a vulnerabilidades comunes (CSRF, inyección SQL, exposición de credenciales).
- Desplegar la aplicación en un entorno de producción accesible desde Internet, demostrando competencia en la configuración de servidores, bases de datos remotas y procesos de despliegue continuo.
- Proporcionar una experiencia de usuario (UX) cuidada, con interfaces responsivas, navegación intuitiva y retroalimentación visual inmediata ante las acciones del usuario.

Público objetivo

El público objetivo de GestiónDocentes está compuesto por los diferentes agentes que intervienen en la gestión académica de un centro de Formación Profesional:

- Equipo directivo (Dirección): responsables de la administración global del centro, la validación de solicitudes, la visualización de estadísticas y la gestión completa del claustro.
- Jefes de departamento: encargados de supervisar a los docentes de su área, validar solicitudes de permisos dentro de su competencia y gestionar los horarios departamentales.
- Profesorado: usuarios finales que consultan su horario semanal, solicitan días de asuntos propios, visualizan el estado de sus peticiones y gestionan su perfil personal.

2. ESTUDIO DE MERCADO

Antes de iniciar el desarrollo de GestiónDocentes se llevó a cabo un análisis de las soluciones existentes en el mercado para la gestión docente en centros educativos. Este estudio de benchmarking permitió identificar las fortalezas y carencias de las alternativas disponibles, y fundamentar las decisiones de diseño adoptadas en el presente proyecto.

Aplicaciones existentes analizadas

Additio App

Additio App es una plataforma de gestión académica orientada principalmente al seguimiento del rendimiento del alumnado. Ofrece cuadernos de notas digitales, rúbricas de evaluación y comunicación con familias. Sin embargo, carece de funcionalidades específicas para la gestión de permisos del profesorado, asignación de guardias o control horario del claustro. Su enfoque es pedagógico, no administrativo, lo que limita su utilidad para la problemática que aborda este proyecto.

Alexia (Educaria)

Alexia es un sistema de gestión académica integral utilizado por numerosos centros concertados y privados. Incluye módulos de gestión de personal, horarios y comunicación. No obstante, se trata de una solución propietaria y de pago cuyo coste de licencia resulta elevado para centros públicos. Además, su arquitectura monolítica dificulta la personalización y la integración con otros sistemas mediante API REST, funcionalidad que GestiónDocentes ofrece de forma nativa.

Horarios Peñalara (generador de horarios)

Esta herramienta de escritorio, muy extendida en centros públicos, se especializa en la generación automática de horarios lectivos. Su fortaleza reside en los algoritmos de optimización combinatoria para cuadrar franjas horarias con restricciones. Sin embargo, no gestiona solicitudes de permisos, guardias ni ofrece un portal web accesible por los docentes. Su alcance es exclusivamente el de la confección de horarios, no la gestión integral del profesorado.

Google Workspace for Education

Numerosos centros utilizan Google Workspace para gestionar comunicación, calendarios y documentos compartidos. Aunque resulta versátil, no está diseñado para flujos de trabajo específicos como la solicitud y aprobación de permisos con reglas de negocio (límite trimestral, cupo diario, prioridad por antigüedad y tipo de funcionario). La gestión de guardias y la trazabilidad de las sustituciones requieren un desarrollo a medida que Google Workspace no proporciona.

Conclusiones del estudio de mercado

Criterio	Additio	Alexia	Peñalara	Google WS	GestiónDoc.
Gestión de permisos	No	Parcial	No	No	Sí, completa
Asignación de guardias	No	No	No	No	Sí, automática
Gestión de horarios	No	Sí	Sí	Parcial	Sí
API REST	Sí	No	No	Sí	Sí, con Swagger
Roles diferenciados	Sí	Sí	No	Parcial	Sí (3 roles)
Software libre	No	No	No	Freemium	Sí
Adaptado a FP pública	No	No	Parcial	No	Sí

Tabla 1. Comparativa de soluciones existentes frente a GestiónDocentes

El análisis comparativo evidencia que ninguna de las soluciones analizadas cubre de forma integrada todas las necesidades identificadas: gestión de permisos con reglas de negocio, asignación automática de guardias, control horario y API REST documentada. GestiónDocentes se posiciona como una solución a medida, de código abierto, diseñada específicamente para centros públicos de Formación Profesional del Principado de Asturias.

3. ANÁLISIS Y DISEÑO DEL SISTEMA

3.1. Requisitos

Requisitos no funcionales

ID	Requisito	Descripción
RNF-01	Rendimiento	El tiempo de respuesta de cualquier operación no debe superar los 3 segundos en condiciones normales de uso.
RNF-02	Seguridad	Las contraseñas se almacenan cifradas con BCrypt. Se implementa protección CSRF en formularios web y se desactiva para la API REST.
RNF-03	Disponibilidad	La aplicación debe estar disponible 24/7 con un tiempo de inactividad máximo planificado del 1 %.
RNF-04	Compatibilidad	La interfaz web debe ser compatible con Chrome, Firefox, Edge y Safari en sus dos últimas versiones estables.
RNF-05	Responsividad	La interfaz debe adaptarse correctamente a resoluciones de escritorio (1920×1080), tablet (768px) y móvil (375px).
RNF-06	Escalabilidad	La arquitectura debe permitir la adición de nuevos módulos sin refactorizar el núcleo existente.
RNF-07	Mantenibilidad	El código sigue el patrón MVC con capas separadas de controlador, servicio y repositorio.
RNF-08	Interoperabilidad	La API REST con formato JSON y documentación OpenAPI 3.0 permite la integración con sistemas

		externos.
RNF-09	Usabilidad	La interfaz utiliza iconografía Bootstrap Icons y retroalimentación visual inmediata (flash messages).
RNF-10	Privacidad	No se exponen datos sensibles en la API (contraseñas, tokens) mediante anotaciones @JsonIgnore y @JsonProperty.

Tabla 2. Requisitos no funcionales del sistema

Requisitos funcionales

ID	Módulo	Requisito	Descripción
RF-01	Autenticación	Inicio de sesión	Los usuarios acceden al sistema mediante correo electrónico y contraseña. Spring Security gestiona la sesión.
RF-02	Autenticación	Cambio obligatorio	Los usuarios creados con contraseña temporal deben cambiarla en su primer inicio de sesión.
RF-03	Autenticación	Recuperación	El sistema genera un token de un solo uso con caducidad de 60 minutos y envía un enlace por correo.
RF-04	Docentes	Listado paginado	Se muestra un listado paginado (10 elementos/página) con filtros por nombre, apellidos y departamento.
RF-05	Docentes	Creación	Los roles Dirección y Jefe de Departamento pueden dar de alta nuevos docentes con validación de formulario.
RF-06	Docentes	Edición	Modificación de los datos de un docente existente, preservando la contraseña y el historial.
RF-07	Docentes	Eliminación	Solo el rol Dirección puede eliminar docentes. La eliminación es transaccional y limpia todas las dependencias.
RF-08	Docentes	Exportación CSV	Generación de un fichero CSV descargable con los datos del claustro completo.
RF-09	Docentes	Perfil	Vista detallada con información personal, historial de solicitudes y tabla de horario semanal.
RF-10	Solicitudes	Solicitud de permiso	Cada docente puede solicitar un día de permiso por trimestre, con descripción y fecha.
RF-11	Solicitudes	Reglas de negocio	El sistema verifica: fecha no pasada, cupo diario (máximo 3) y límite trimestral (1 por trimestre).

RF-12	Solicitudes	Validación	Dirección y Jefes de Departamento aprueban o rechazan solicitudes. Se notifica por correo al docente.
RF-13	Solicitudes	Priorización	Las solicitudes se ordenan por tipo de funcionario, antigüedad y nota de oposición.
RF-14	Guardias	Panel de ausencias	Calendario mensual con los días de ausencias aprobadas y los docentes afectados.
RF-15	Guardias	Asignación automática	El algoritmo prioriza docentes del mismo departamento y selecciona al de menos guardias realizadas.
RF-16	Horarios	Visualización	Cada docente visualiza su horario en formato rejilla (lunes a viernes, 7 franjas) con asignatura y aula.
RF-17	Horarios	Gestión	Dirección y Jefes de Departamento asignan franjas horarias indicando día, hora, asignatura y aula.
RF-18	Estadísticas	Panel	Solo accesible por Dirección. Muestra totales de solicitudes, aprobadas, rechazadas y datos departamentales.
RF-19	API REST	Endpoints	La API expone endpoints GET para docentes, departamentos, horarios y solicitudes con paginación y filtros.
RF-20	API REST	Documentación	Swagger UI disponible en /swagger-ui.html permite probar los endpoints desde el navegador.
RF-21	Dashboard	Diferenciado	El panel principal muestra tarjetas y datos adaptados al perfil del usuario autenticado.

Tabla 3. Requisitos funcionales del sistema

3.2. Casos de uso

A continuación se describen los casos de uso principales del sistema, organizados por actor. Cada caso de uso detalla el flujo básico de interacción entre el usuario y la aplicación.

Actor: Profesor

ID	Caso de uso	Actor	Descripción	Precondición	Postcondición
CU-01	Iniciar sesión	Profesor	El docente accede a /login, introduce su correo y contraseña, y el sistema valida las credenciales mediante Spring Security.	Correo y contraseña válidos	Sesión iniciada, dashboard visible
CU-02	Consultar horario	Profesor	El docente accede a «Mi Horario» y visualiza una rejilla semanal con las asignaturas, aulas y franjas horarias asignadas.	Sesión activa	Horario semanal mostrado
CU-03	Solicitar permiso	Profesor	El docente selecciona una fecha futura, añade una descripción y envía la solicitud. El sistema valida las reglas de negocio.	Sesión activa, cupo no agotado	Solicitud registrada como pendiente
CU-04	Consultar solicitudes	Profesor	El docente visualiza el historial completo de sus solicitudes con estado y estadísticas personales.	Sesión activa	Listado de solicitudes mostrado
CU-05	Cambiar contraseña	Profesor	El docente solicita un enlace de recuperación por correo, recibe un token de un solo uso y establece una nueva contraseña.	Correo registrado en el sistema	Contraseña actualizada

Tabla 4. Casos de uso del actor Profesor

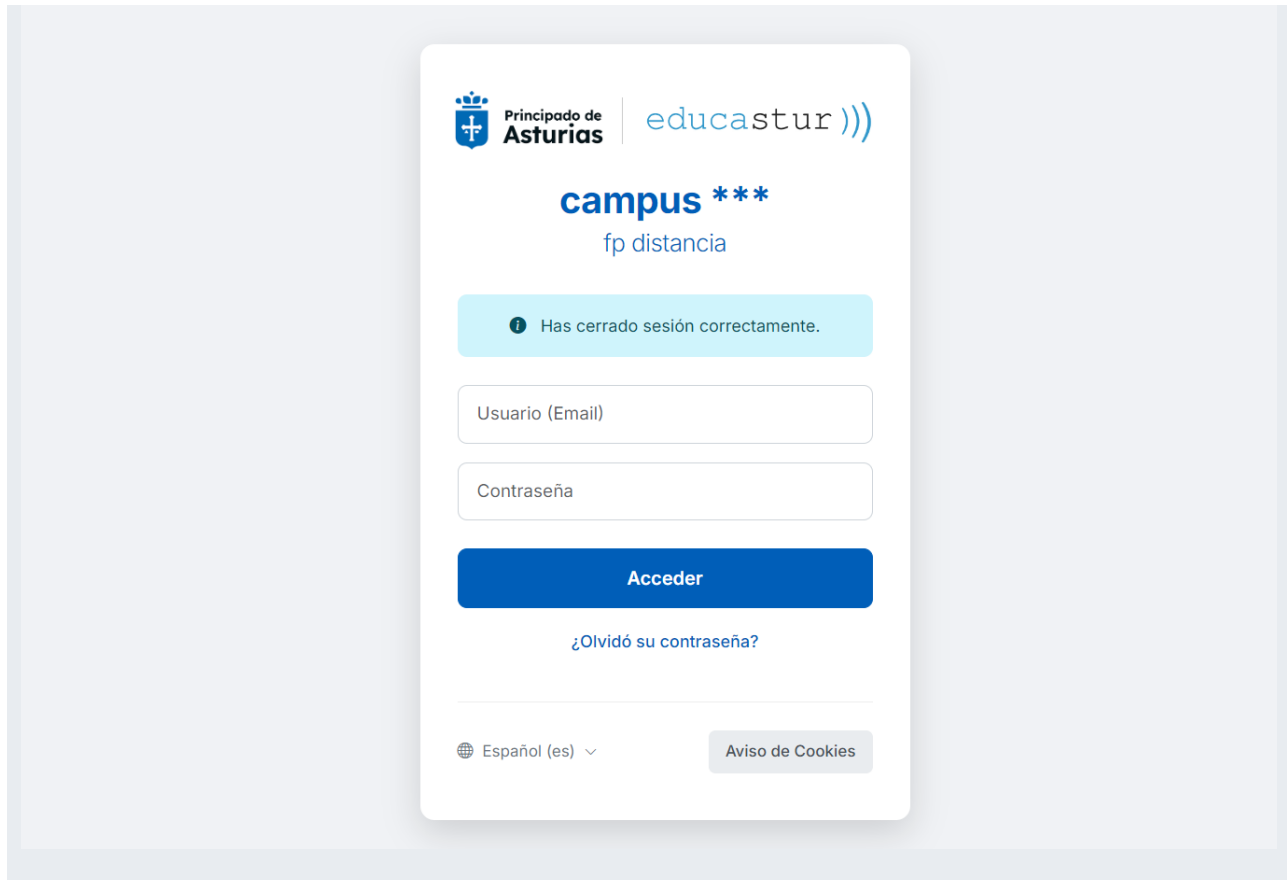


Figura 2. Pantalla de inicio de sesión de la aplicación

Actor: Jefe de Departamento

ID	Caso de uso	Actor	Descripción	Precondición	Postcondición
CU-06	Validar solicitudes	Jefe Dept.	El jefe de departamento accede al panel de administración y aprueba o rechaza solicitudes pendientes.	Sesión con rol Jefe de Dept.	Solicitud actualizada, correo enviado
CU-07	Crear docente	Jefe Dept.	Se rellena el formulario con datos personales, departamento y rol. El sistema genera contraseña temporal.	Sesión con rol Jefe de Dept.	Docente creado
CU-08	Editar docente	Jefe Dept.	Se modifican los datos de un docente existente. La contraseña y el historial se	Sesión activa, docente existente	Datos actualizados

			preservan.		
CU-09	Gestionar horarios	Jefe Dept.	Se asignan franjas horarias a docentes seleccionando día, hora, asignatura y aula.	Sesión con rol Jefe de Dept.	Entrada de horario guardada

Tabla 5. Casos de uso del actor Jefe de Departamento

Actor: Dirección (Administrador)

ID	Caso de uso	Actor	Descripción	Precondición	Postcondición
CU-10	Eliminar docente	Dirección	Eliminación transaccional de un docente, incluyendo la limpieza de todas las dependencias asociadas.	Sesión con rol Dirección	Docente y datos asociados eliminados
CU-11	Consultar estadísticas	Dirección	Visualización del panel con indicadores globales: total de solicitudes, aprobadas, rechazadas y datos departamentales.	Sesión con rol Dirección	Panel de estadísticas mostrado
CU-12	Asignar guardia	Dirección	Selección de un docente ausente. El sistema ejecuta el algoritmo de asignación y muestra el sustituto.	Ausencia aprobada sin sustituto	Sustituto asignado
CU-13	Exportar claustro	Dirección	Descarga de un fichero CSV con los datos de todos los docentes del centro.	Sesión activa	Fichero CSV descargado
CU-14	Acceder a la API	Dirección	Consulta de la documentación interactiva en /swagger-ui.html y consumo de los endpoints.	Sesión con rol Dirección	Datos JSON obtenidos

Tabla 6. Casos de uso del actor Dirección

3.3. Diseño visual

Guía de estilos (UI)

El diseño visual de GestiónDocentes sigue un enfoque minimalista y profesional, inspirado en las tendencias actuales de diseño de aplicaciones SaaS empresariales. Se ha optado por una estética limpia que prioriza la legibilidad y la eficiencia en la consulta de información.

Paleta de colores:

Nombre	Código hexadecimal	Uso
Primary (Indigo)	#4F46E5	Color principal de la marca, botones primarios, sidebar
Primary Dark	#3730A3	Gradientes, estado hover sobre elementos primarios
Background	#F1F5F9	Fondo general de la aplicación (gris azulado claro)
Surface	#FFFFFF	Tarjetas, modales, sidebar, tablas
Text Primary	#1E293B	Texto principal, títulos, etiquetas
Text Secondary	#64748B	Texto secundario, descripciones, metadatos
Success	#22C55E	Indicadores de aprobación, badges de estado positivo
Warning	#F59E0B	Indicadores de pendiente, alertas informativas
Danger	#EF4444	Indicadores de rechazo, acciones destructivas
Border	#E2E8F0	Bordes de tarjetas, separadores

Tabla 7. Paleta de colores de la interfaz

Tipografía:

- Familia tipográfica principal: Inter (Google Fonts), con fallback a system-ui y sans-serif.
- Tamaño base del cuerpo de texto: 1rem (16px).
- Pesos utilizados: 300 (light), 400 (regular), 600 (semibold), 700 (bold).
- Interlineado de interfaz: 1,5 para cuerpo de texto, 1,2 para títulos y encabezados.

Componentes visuales:

- Tarjetas (cards): bordes redondeados de 12px, sombra sutil, sin borde visible.
- Botones primarios: fondo índigo (#4F46E5), texto blanco, padding 0,6rem × 1,2rem.
- Badges de estado: colores semánticos con fondo semitransparente y borde fino.
- Avatares: círculos de 35px con iniciales del usuario, fondo primario, texto blanco.
- Iconografía: Bootstrap Icons 1.11.0, tamaño estándar 1,2rem en sidebar.



Figura 3. Sidebar de navegación y cabecera con perfil de usuario

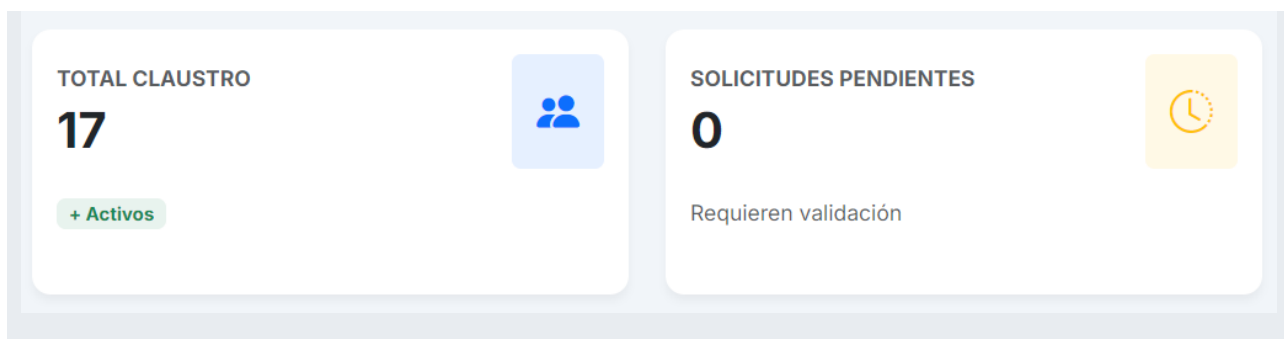


Figura 4. Ejemplo de tarjetas del dashboard con estadísticas

Estructura de navegación y UX

La estructura de navegación sigue un patrón de sidebar fija a la izquierda (260px de ancho) con área de contenido principal a la derecha. Este patrón es ampliamente reconocido en aplicaciones de gestión y proporciona un acceso rápido y predecible a todas las secciones del sistema.

Mapa del sitio:

- Dashboard (/) — Panel principal con contenido diferenciado por rol
- Claustro (/web/docentes) — Listado paginado con búsqueda y filtros
 - Crear docente (/web/docentes/crear) [Dirección, Jefe Dept.]
 - Editar docente (/web/docentes/editar/{id}) [Dirección, Jefe Dept.]
 - Perfil de docente (/web/docentes/perfil/{id})
 - Exportar CSV (/web/docentes/exportar)
- Guardias (/web/guardias/panel) — Calendario de ausencias y asignación
- Mi Horario (/web/horarios) — Rejilla semanal del docente autenticado
- Solicitar Permiso (/web/solicitud/nueva) — Formulario de nueva solicitud
- Mis Solicitudes (/web/solicitudes/mis-solicitudes/{id}) — Historial personal
- Administración (/web/admin/validar) — Panel de validación [Dirección, Jefe Dept.]
- Gestionar Horarios (/web/horarios/gestionar) [Dirección, Jefe Dept.]
- Estadísticas (/web/admin/estadisticas) [Solo Dirección]
- API Docs (/swagger-ui.html) — Documentación interactiva [Solo Dirección]
- Login (/login) — Formulario de inicio de sesión
- Recuperar contraseña (/auth/forgot-password)

Flujo de interacción del usuario:

Al acceder a la aplicación, el usuario no autenticado es redirigido automáticamente a la página de login. Tras introducir credenciales válidas, se verifica si debe cambiar la contraseña temporal; en caso afirmativo, se redirige al formulario de cambio obligatorio. Si la contraseña ya fue actualizada, se presenta el dashboard con contenido adaptado al rol. La barra lateral muestra únicamente las opciones accesibles según el perfil del usuario autenticado, evitando confusiones y proporcionando una experiencia limpia y coherente.

Cada acción del usuario (creación, edición, eliminación, aprobación, rechazo) genera un mensaje de retroalimentación visual (flash message) que confirma el resultado de la operación. Los errores de validación se muestran en línea junto al campo afectado, y las acciones destructivas requieren confirmación mediante diálogo modal de JavaScript.

3.4. Modelo de clases/datos

Modelo de datos

La base de datos del sistema, denominada gestion_docentes, está alojada en un servidor MySQL 8 y se gestiona mediante JPA/Hibernate con la estrategia de generación de esquema ddl-auto=update. El modelo relacional se compone de 9 tablas principales, diseñadas siguiendo los principios de normalización hasta la Tercera Forma Normal (3FN) para eliminar redundancias y garantizar la integridad referencial.

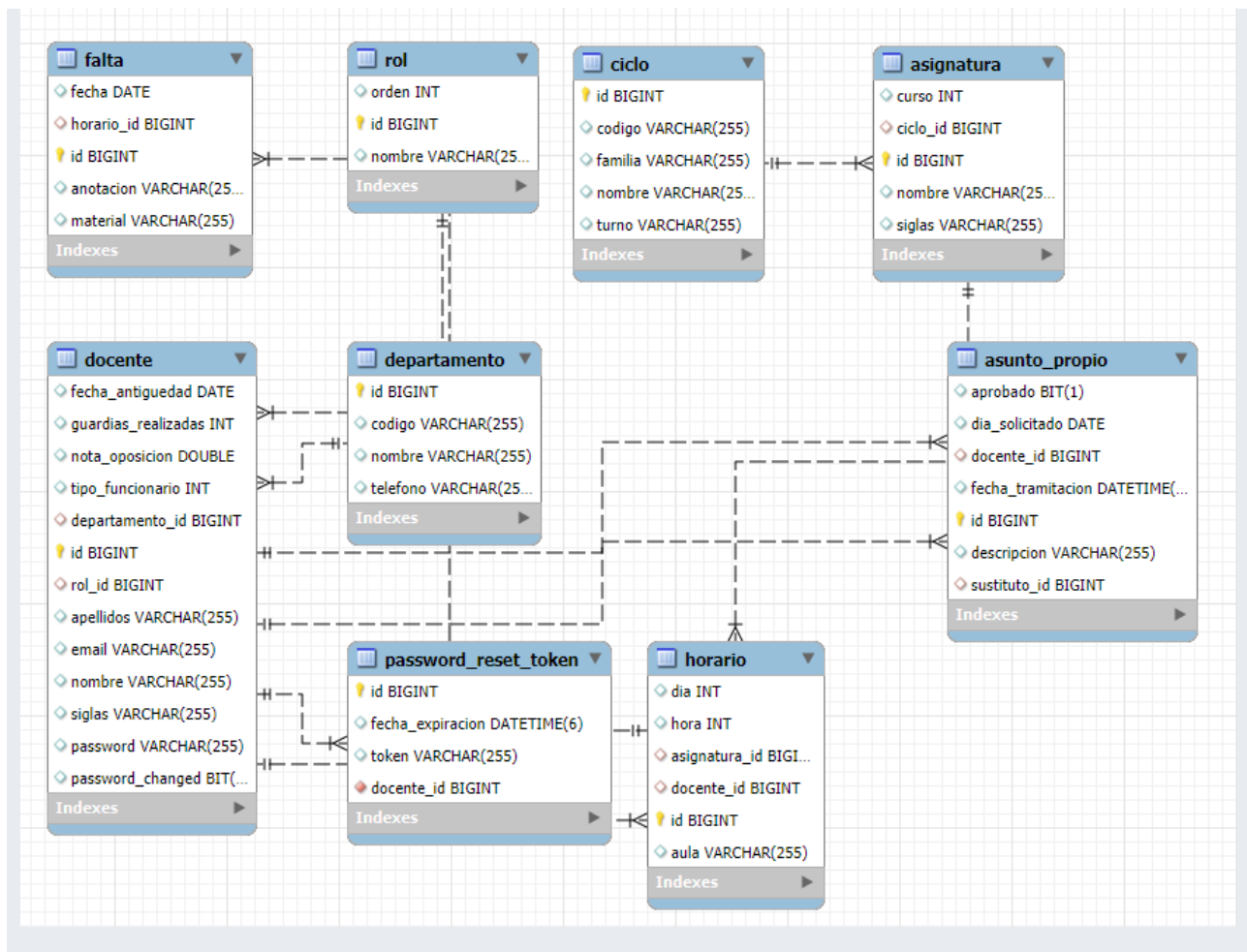


Figura 5. Diagrama Entidad-Relación de la base de datos (generado con MySQL Workbench)

Relaciones principales del modelo:

- Departamento (1) $\longrightarrow$ (N) Docente: cada docente pertenece a un departamento.
- Rol (1) $\longrightarrow$ (N) Docente: cada docente tiene asignado un único rol.

- Docente (1) —→ (N) AsuntoPropio: un docente puede tener múltiples solicitudes de permisos.
- Docente (1) —→ (N) AsuntoPropio (como sustituto): un docente puede ser sustituto en varias ausencias.
- Docente (1) —→ (N) Horario: un docente tiene múltiples franjas horarias asignadas.
- Asignatura (1) —→ (N) Horario: una asignatura se imparte en múltiples franjas horarias.
- Ciclo (1) —→ (N) Asignatura: un ciclo formativo agrupa múltiples asignaturas.
- Horario (1) —→ (N) Falta: cada franja horaria puede registrar múltiples faltas.
- Docente (1) —→ (1) PasswordResetToken: token temporal para restablecimiento de contraseña.

Diccionario de datos

El diccionario de datos es una guía que describe las tablas de la base de datos, los campos que contiene cada una, el significado de cada campo y el tipo de dato utilizado. Su propósito es facilitar la comprensión del modelo de datos, documentar el proyecto y permitir a otros desarrolladores trabajar con la base de datos de forma autónoma.

Tabla docente:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único del docente	PK
nombre	VARCHAR(255)	Nombre del docente	
apellidos	VARCHAR(255)	Apellidos del docente	
email	VARCHAR(255)	Correo electrónico institucional (único)	
siglas	VARCHAR(5)	Siglas identificativas del docente	
password	VARCHAR(255)	Contraseña cifrada con BCrypt	
tipo_funcionario	INT	Tipo de funcionario: 1=Carrera, 2=Prácticas, 3=Interino	

fecha_antigüedad	DATE	Fecha de antigüedad en el centro	
nota_oposicion	DOUBLE	Nota obtenida en la oposición	
password_changed	BOOLEAN	Indica si el docente ya cambió la contraseña temporal	
guardias_realizadas	INT	Número de guardias realizadas en el curso actual	
departamento_id	BIGINT	Departamento al que pertenece el docente	FK
rol_id	BIGINT	Rol asignado al docente en el sistema	FK

Tabla 8. Diccionario de datos — Tabla docente

Tabla departamento:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único del departamento	PK
nombre	VARCHAR(255)	Nombre completo del departamento	
codigo	VARCHAR(255)	Código abreviado del departamento (ej.: IFC, ADG)	
telefono	VARCHAR(255)	Teléfono de contacto del departamento	

Tabla 9. Diccionario de datos — Tabla departamento

Tabla rol:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único del rol	PK
nombre	VARCHAR(255)	Nombre del rol (Dirección, Jefe de Departamento,	

		Profesor)	
orden	INT	Orden de prioridad del rol para la presentación visual	

Tabla 10. Diccionario de datos — Tabla rol

Tabla asunto_propio:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único de la solicitud	PK
dia_solicitado	DATE	Fecha del día de permiso solicitado	
descripcion	VARCHAR(255)	Motivo o descripción de la solicitud	
fecha_tramitacion	DATETIME	Fecha y hora en que se registró la solicitud	
aprobado	BOOLEAN	Estado: NULL=pendiente, TRUE=aprobada, FALSE=rechazada	
docente_id	BIGINT	Docente que realiza la solicitud	FK
sustituto_id	BIGINT	Docente asignado como sustituto para cubrir la ausencia	FK

Tabla 11. Diccionario de datos — Tabla asunto_propio

Tabla horario:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único de la entrada de horario	PK
dia	INT	Día de la semana (1=Lunes, 2=Martes, ..., 5=Viernes)	
hora	INT	Franja horaria dentro del	

		día (1 a 7)	
aula	VARCHAR(255)	Aula donde se imparte la clase	
docente_id	BIGINT	Docente asignado a esta franja horaria	FK
asignatura_id	BIGINT	Asignatura impartida en esta franja	FK

Tabla 12. Diccionario de datos — Tabla horario

Tabla asignatura:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único de la asignatura	PK
nombre	VARCHAR(255)	Nombre completo de la asignatura	
siglas	VARCHAR(255)	Siglas de la asignatura (ej.: DWES, DIW)	
curso	INT	Curso al que pertenece la asignatura (1 o 2)	
ciclo_id	BIGINT	Ciclo formativo al que pertenece la asignatura	FK

Tabla 13. Diccionario de datos — Tabla asignatura

Tabla ciclo:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único del ciclo formativo	PK
nombre	VARCHAR(255)	Nombre del ciclo (ej.: DAW, DAM, ASIR)	
familia	VARCHAR(255)	Familia profesional a la que pertenece	
codigo	VARCHAR(255)	Código oficial del ciclo formativo	

turno	VARCHAR(255)	Turno de impartición (Diurno, Vespertino)	
-------	--------------	--	--

Tabla 14. Diccionario de datos — Tabla ciclo

Tabla falta:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único del registro de falta	PK
fecha	DATE	Fecha en la que se produjo la ausencia	
anotacion	VARCHAR(255)	Anotación del sustituto sobre cómo se desarrolló la guardia	
material	VARCHAR(255)	Material dejado por el docente ausente para la guardia	
horario_id	BIGINT	Franja horaria afectada por la ausencia	FK

Tabla 15. Diccionario de datos — Tabla falta

Tabla password_reset_token:

Campo	Tipo	Descripción	Clave
id	BIGINT	Identificador único del token de restablecimiento	PK
token	VARCHAR(255)	Token UUID generado aleatoriamente para verificación	
fecha_expiracion	DATETIME	Fecha y hora de expiración del token (60 minutos)	
docente_id	BIGINT	Docente que solicitó el restablecimiento de contraseña	FK

Tabla 16. Diccionario de datos — Tabla password_reset_token

3.5. Diseño de la API

GestiónDocentes expone una API REST bajo el prefijo `/api/` que permite el acceso programático a los datos del sistema. La API sigue los principios RESTful: recursos identificados por URI, operaciones HTTP semánticas (GET, DELETE), respuestas en formato JSON y códigos de estado HTTP estándar. La documentación interactiva se genera automáticamente mediante `springdoc-openapi 2.5.0` y es accesible en `/swagger-ui.html`.

Autenticación de la API:

Todos los endpoints bajo `/api/**` requieren autenticación. La API comparte la sesión HTTP del navegador gestionada por Spring Security. Las peticiones no autenticadas reciben un código 401 Unauthorized. La protección CSRF se ha desactivado exclusivamente para los endpoints de la API para permitir el consumo desde clientes REST externos (Postman, aplicaciones móviles).

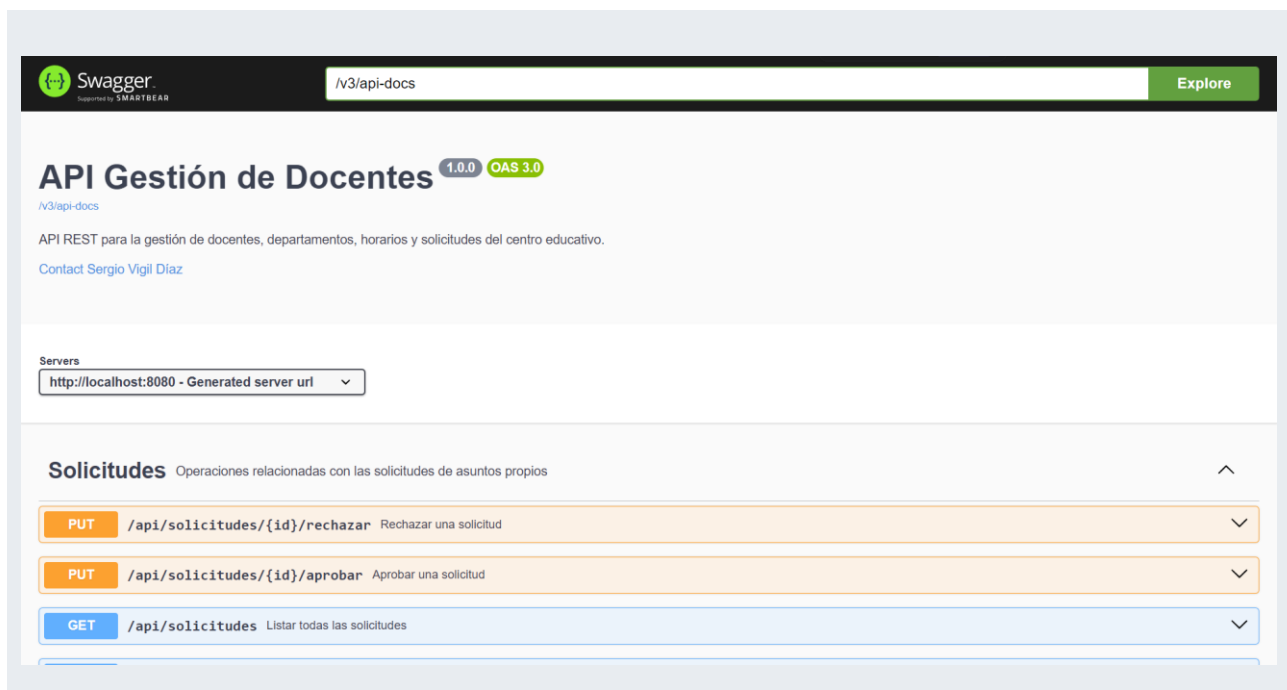


Figura 6. Documentación interactiva de la API en Swagger UI

Endpoints de Docentes (`/api/docentes`)

Método	Endpoint	Descripción	Parámetros	Respuesta
GET	<code>/api/docentes</code>	Listar todos los docentes	page, size, sort	200 OK — Page<Docente>

		(paginado)		
GET	/api/docentes/{id}	Obtener un docente por su identificador	id (path)	200 OK / 404 Not Found
GET	/api/docentes/departamento/{nombre}	Listar docentes de un departamento	nombre (path)	200 OK — List<Docente>
GET	/api/docentes/buscar?q=...	Buscar docentes por nombre o apellidos	q (query)	200 OK — List<Docente>
DELETE	/api/docentes/{id}	Eliminar un docente	id (path)	204 No Content / 404

Tabla 17. Endpoints de la API — Recurso Docentes

Endpoints de Solicitudes (/api/solicitudes)

Método	Endpoint	Descripción	Parámetros	Respuesta
GET	/api/solicitudes	Listar todas las solicitudes	Ninguno	200 OK — List<AsuntoPropio>
GET	/api/solicitudes/{id}	Obtener una solicitud por ID	id (path)	200 OK / 404 Not Found
GET	/api/solicitudes/docente/{id}	Listar solicitudes de un docente	id (path)	200 OK — List<AsuntoPropio>
GET	/api/solicitudes/pendientes	Listar solicitudes pendientes	Ninguno	200 OK — List<AsuntoPropio>

Tabla 18. Endpoints de la API — Recurso Solicitudes

Endpoints de Departamentos (/api/departamentos)

Método	Endpoint	Descripción	Parámetros	Respuesta
GET	/api/departamentos	Listar todos los departamentos	Ninguno	200 OK — List<Departamento>
GET	/api/departamentos/{id}	Obtener un departamento por ID	id (path)	200 OK / 404 Not Found

Tabla 19. Endpoints de la API — Recurso Departamentos

Endpoints de Horarios (/api/horarios)

Método	Endpoint	Descripción	Parámetros	Respuesta
GET	/api/horarios	Listar todos los horarios	Ninguno	200 OK — List<Horario>
GET	/api/horarios/docente/{id}	Listar horarios de un docente	id (path)	200 OK — List<Horario>

Tabla 20. Endpoints de la API — Recurso Horarios

4. DESARROLLO TÉCNICO

4.1. Backend (Spring Boot)

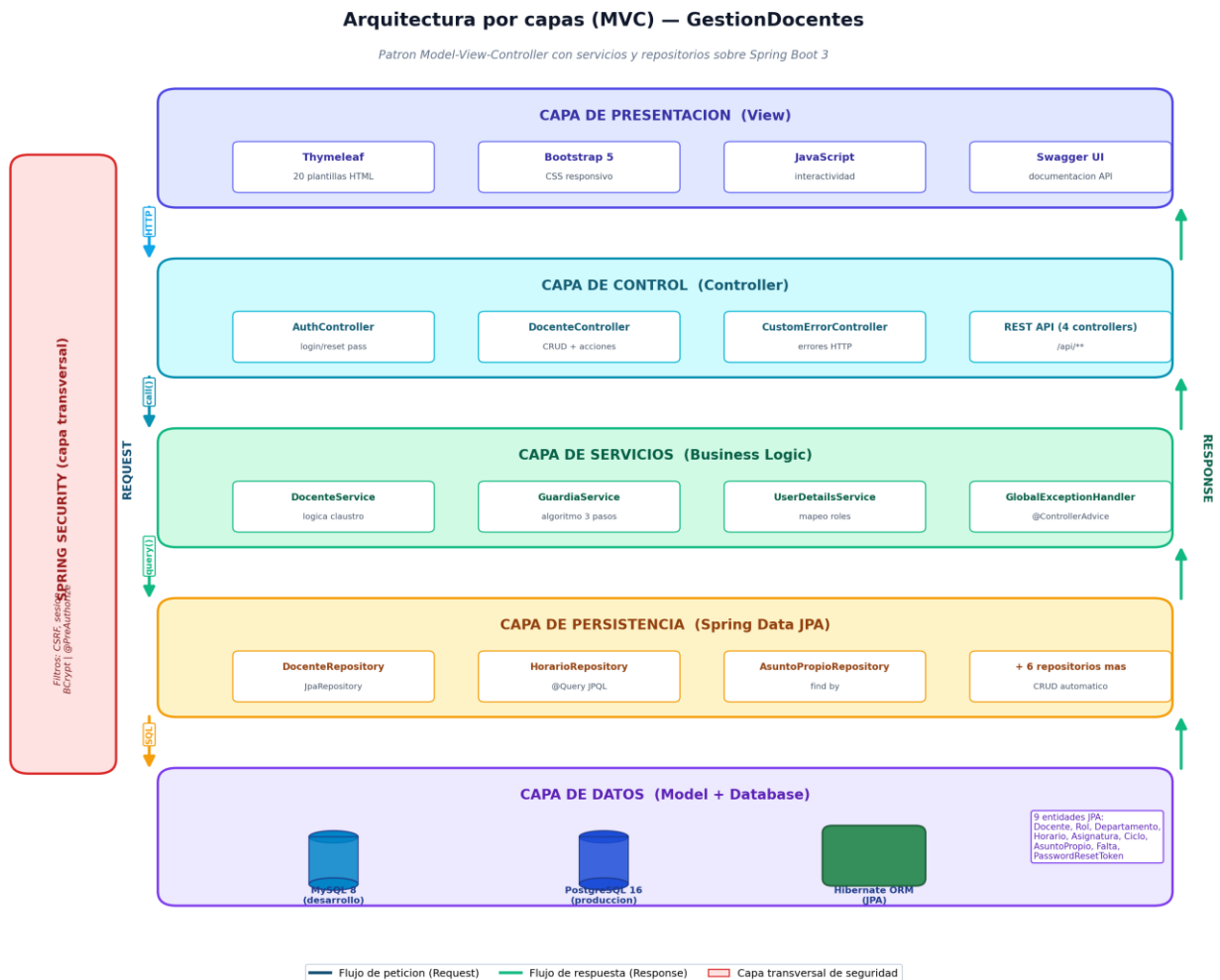


Figura 15. Arquitectura por capas (MVC) del proyecto.

El backend de GestiónDocentes está construido con Spring Boot 3.3.2 sobre Java 21. Se ha elegido este framework por su madurez, su ecosistema extenso y su capacidad para integrar de forma coherente los múltiples módulos necesarios: persistencia (JPA), seguridad (Spring Security), renderizado de vistas (Thymeleaf), API REST y documentación (OpenAPI).

Arquitectura por capas:

El proyecto sigue una arquitectura en capas claramente delimitadas, lo que facilita la separación de responsabilidades, la testabilidad y el mantenimiento a largo plazo:

- Capa de presentación (Controller): gestiona las peticiones HTTP entrantes, tanto web (DocenteController, AuthController) como REST (DocenteApiController, SolicitudApiController, DepartamentoApiController, HorarioApiController).
- Capa de servicio (Service): contiene la lógica de negocio encapsulada en interfaces e implementaciones (DocenteServiceImpl, GuardiaService, UserDetailsServiceImpl). El servicio de guardias implementa un algoritmo de asignación con doble criterio.
- Capa de acceso a datos (Repository): interfaces que extienden JpaRepository de Spring Data, proporcionando operaciones CRUD automáticas y consultas personalizadas mediante @Query JPQL.
- Capa de modelo (Entity): clases JPA anotadas que mapean directamente a las tablas de la base de datos MySQL. Se utiliza Lombok (@Data) para reducir el código repetitivo.

Seguridad:

Spring Security gestiona la autenticación y autorización del sistema. La configuración se centraliza en SecurityConfig.java, donde se definen las reglas de acceso por URL y se habilita la seguridad a nivel de método con @EnableMethodSecurity. Los tres roles del sistema se mapean desde la base de datos en UserDetailsServiceImpl: «Dirección» → ROLE_ADMIN, «Jefe de Departamento» → ROLE_JEFE_DEPT y «Profesor» → ROLE_USER.

- Cifrado de contraseñas: BCryptPasswordEncoder con factor de coste por defecto (10 rondas).
- Protección CSRF: activa para formularios web, desactivada para /api/** para consumo REST externo.
- Control de acceso: @PreAuthorize a nivel de método para operaciones sensibles.
- Gestión de sesiones: sesión HTTP con cookie JSESSIONID, login personalizado con el correo como username.

Validación:

Se utiliza Bean Validation (Jakarta Validation) con las anotaciones @NotBlank, @Email, @Size y @Valid en combinación con BindingResult para validar los formularios del lado del servidor. Los errores se muestran en línea en la vista Thymeleaf mediante th:errors.

Dependencias principales (pom.xml):

Dependencia	Versión	Propósito
spring-boot-starter-data-jpa	3.3.2	Persistencia JPA con Hibernate

spring-boot-starter-web	3.3.2	Servidor web embebido (Tomcat), MVC
spring-boot-starter-security	3.3.2	Autenticación y autorización
spring-boot-starter-validation	3.3.2	Bean Validation (Jakarta)
spring-boot-starter-thymeleaf	3.3.2	Motor de plantillas server-side
spring-boot-starter-mail	3.3.2	Envío de correos electrónicos SMTP
thymeleaf-extras-springsecurity6	—	Integración Thymeleaf con Spring Security
springdoc-openapi-starter-webmvc-ui	2.5.0	Documentación OpenAPI/Swagger UI
mysql-connector-j	9.1.0	Driver JDBC para MySQL 8
lombok	—	Reducción de código boilerplate
modelmapper	3.2.0	Mapeo entre DTO y entidades
spring-boot-devtools	3.3.2	Recarga automática en desarrollo

Tabla 21. Dependencias principales del proyecto

4.2. Frontend

La capa de presentación de GestiónDocentes se construye con Thymeleaf 3, un motor de plantillas del lado del servidor que genera HTML dinámico. Se optó por esta tecnología en lugar de un framework SPA (React, Angular, Vue) por las siguientes razones:

- Integración nativa con Spring Boot: Thymeleaf forma parte del ecosistema Spring y se configura con una sola dependencia.
- Seguridad simplificada: al renderizar en el servidor, no se expone lógica de negocio al cliente.
- SEO y rendimiento inicial: las páginas se entregan completamente renderizadas, lo que mejora el tiempo de primera pintura.

Tecnologías del frontend:

Tecnología	Versión	Propósito
Thymeleaf	3.x	Motor de plantillas server-side con layout dialect
Bootstrap	5.3.0	Framework CSS para componentes responsivos y grid system
Bootstrap Icons	1.11.0	Biblioteca de iconos SVG integrados como fuente web
Google Fonts (Inter)	—	Tipografía moderna y legible para interfaces
JavaScript nativo	ES6+	Diálogos de confirmación, interacciones puntuales

Tabla 22. Tecnologías del frontend

Estructura de plantillas:

Las vistas se organizan siguiendo un sistema de layout maestro (layout/main.html) que define la estructura común (sidebar, header) y un fragmento de contenido reemplazable. Cada página hereda de este layout mediante th:replace. Esta arquitectura evita la duplicación de código HTML y garantiza la coherencia visual en toda la aplicación.

Plantillas del sistema (20 vistas):

- layout/main.html — Layout maestro con sidebar, header y área de contenido

- layout/auth-layout.html — Layout para páginas de autenticación (sin sidebar)
- index.html — Dashboard principal con contenido diferenciado por rol
- auth/login.html — Formulario de inicio de sesión
- auth/forgot-password.html — Solicitud de recuperación de contraseña
- auth/change-password.html — Formulario de cambio de contraseña
- docentes/list.html — Listado paginado con búsqueda, filtros y acciones CRUD
- docentes/form.html — Formulario dual (creación/edición) con validación inline
- docentes/perfil.html — Vista de perfil con horario y solicitudes
- solicitudes/form.html — Formulario de nueva solicitud con reglas de negocio
- solicitudes/my-list.html — Historial personal de solicitudes
- solicitudes/admin-list.html — Panel de validación para administradores
- horarios/ver.html — Rejilla semanal del horario del docente
- horarios/gestionar.html — Formulario de asignación de franjas horarias
- guardias/panel.html — Calendario mensual de ausencias
- guardias/resultado.html — Resultado de asignación de guardia
- admin/stats.html — Panel de estadísticas globales
- error/404.html — Página personalizada de error 404
- error/500.html — Página personalizada de error 500
- error/acceso-denegado.html — Página de acceso denegado (403)

Claustro de Profesores
Listado completo del personal docente del centro.

Exportar CSV | Nuevo Docente

Buscar docente... | Todos los Dept. ▼

DOCENTE	DEPARTAMENTO	SIGLAS	ROL	ACCIONES
DÁ Álvarez Rubio, David davidar@educastur.org	IFC Informática y Comunicaciones	ARDa	-	⋮
JC Campos Rubio, Javier javiercr@educastur.org	FME Fabricación Mecánica	CRJa	-	⋮
DF Fernández Álvarez, Diego diegofa@educastur.org	ELE Electricidad y Electrónica	FADi	-	⋮
HG García Prado, Hugo hugogp@educastur.org	IMA Instalación y Mantenimiento	GPHu	-	⋮
LG González Pérez, Laura				⋮

Figura 7. Listado de docentes con paginación, búsqueda y filtros por departamento

Registrar Nuevo Docente

Se generará una contraseña temporal: **1234**

Nombre

Apellidos

Correo Electrónico

Siglas

Departamento

Informática y Comunicaciones

▼

Rol en el Centro

Dirección

▼

Tipo Funcionario

1 - Carrera

▼

Nota Oposición

Guardar Docente

Figura 8. Formulario de creación/edición de docente con validación inline

GestionApp

PRINCIPAL

Dashboard
Claustro
Guardias

GESTIÓN

Solicitar Permiso
Mis Solicitudes
Administración
Cerrar Sesión

DA

David Alvarez

Sin rol Sin departamento Interino

david@educastur.org

0

Guardias realizadas

0

Solicitudes aprobadas

0

Solicitudes pendientes

0

Solicitudes rechazadas

Datos personales

NOMBRE COMPLETO

David Alvarez

SIGLAS

DA

CORREO ELECTRÓNICO

david@educastur.org

DEPARTAMENTO

—

ROL

—

TIPO DE FUNCIONARIO

Interino

FECHA DE ANTIGÜEDAD

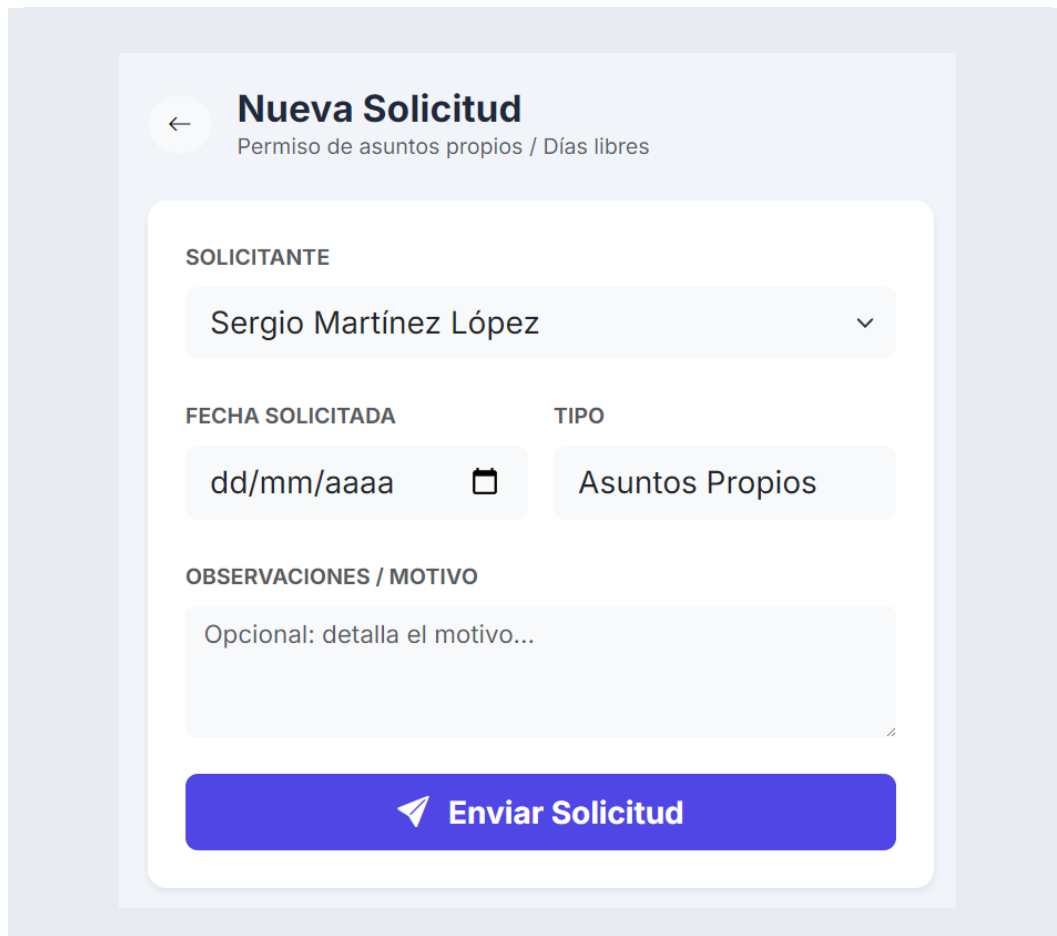
—

Horario semanal

Hora	Lunes	Martes	Miércoles	Jueves	Viernes
1ª	IGR 803	LMR 803	—	—	IGR 803
2ª	—	LMR 803	IGR 803	IGR 803	—
3ª	—	IGR 803	—	—	—
4ª	LMR 803	—	LMR 803	LMR 803	—
5ª	LMR 803	—	—	IGR 803	—
6ª	—	—	IGR 803	IGR 803	IGR 803

Figura 9. Perfil detallado de un docente con horario semanal y solicitudes

30



← **Nueva Solicitud**
Permiso de asuntos propios / Días libres

SOLICITANTE

Sergio Martínez López

FECHA SOLICITADA **TIPO**

dd/mm/aaaa  Asuntos Propios

OBSERVACIONES / MOTIVO

Opcional: detalla el motivo...

 **Enviar Solicitud**

Figura 10. Formulario de solicitud de día de asuntos propios



GestionApp

Bienvenido de nuevo Administrador 

Administración de Solicitudes
Valida y gestiona las peticiones pendientes.

ADM Administrador Principal
Dirección

Exportar Filtrar

DOCENTE	DEPARTAMENTO	FECHA SOLICITADA	PRIORIDAD (ALGORITMO)	ESTADO	ACCIONES
 Sergio Martinez sergio@educastur.org		 2026-05-18	Carrera ★ 9.5	Pendiente	 

PRINCIPAL

- Dashboard
- Claustro
- Guardias

GESTIÓN

- Solicitar Permiso
- Mis Solicitudes
- Administración**
- Cerrar Sesión

Figura 11. Panel de validación de solicitudes (vista administrador)

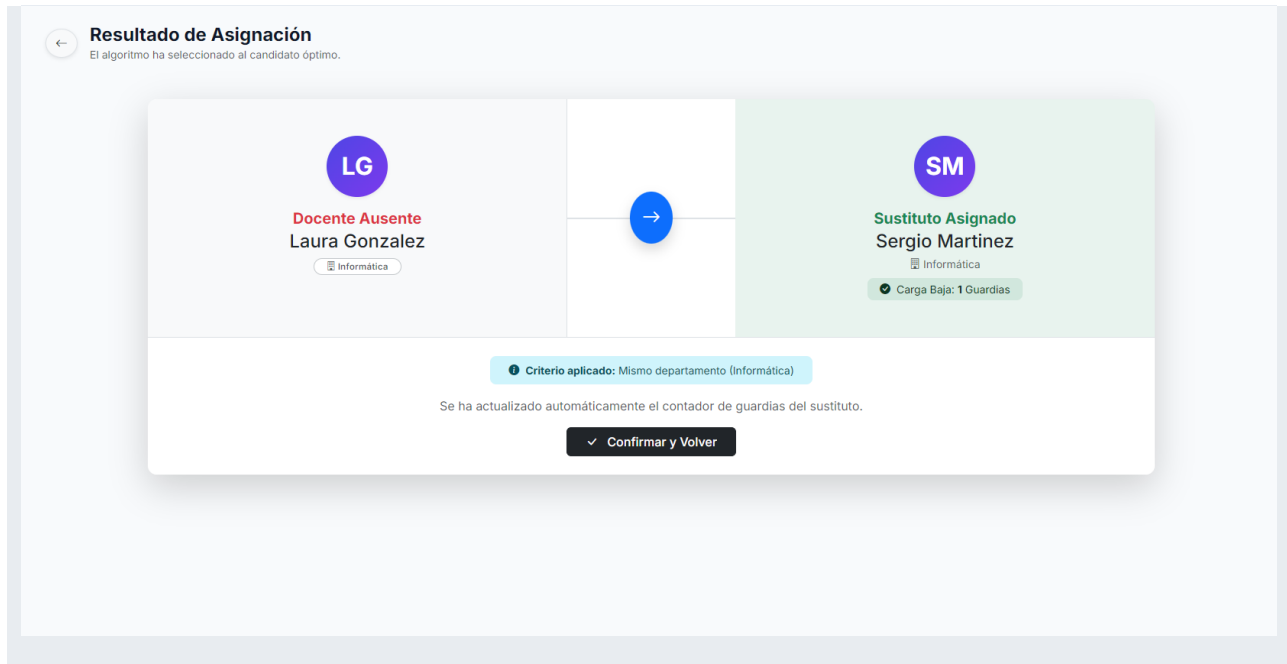


Figura 12. Calendario de guardias y asignación automática de sustitutos

Mi Horario Semanal
Vista de tu horario lectivo semanal completo.

ADM Administrador Principal Informática

HORA	LUNES	MARTES	MIÉRCOLES	JUEVES	VIERNES
1ª	RDL B01	RDL B01	PRG B01	PRG B01	-
2ª	RDL B01	-	-	PRG B01	RDL B01
3ª	-	RDL B01	PRG B01	RDL B01	RDL B01
4ª	PRG B01	PRG B01	PRG B01	-	RDL B01
5ª	-	-	-	-	PRG B01
6ª	-	-	RDL B01	PRG B01	-
7ª	-	-	-	RDL B01	PRG B01

● Clase asignada - Hora libre

Figura 13. Rejilla de horario semanal del docente

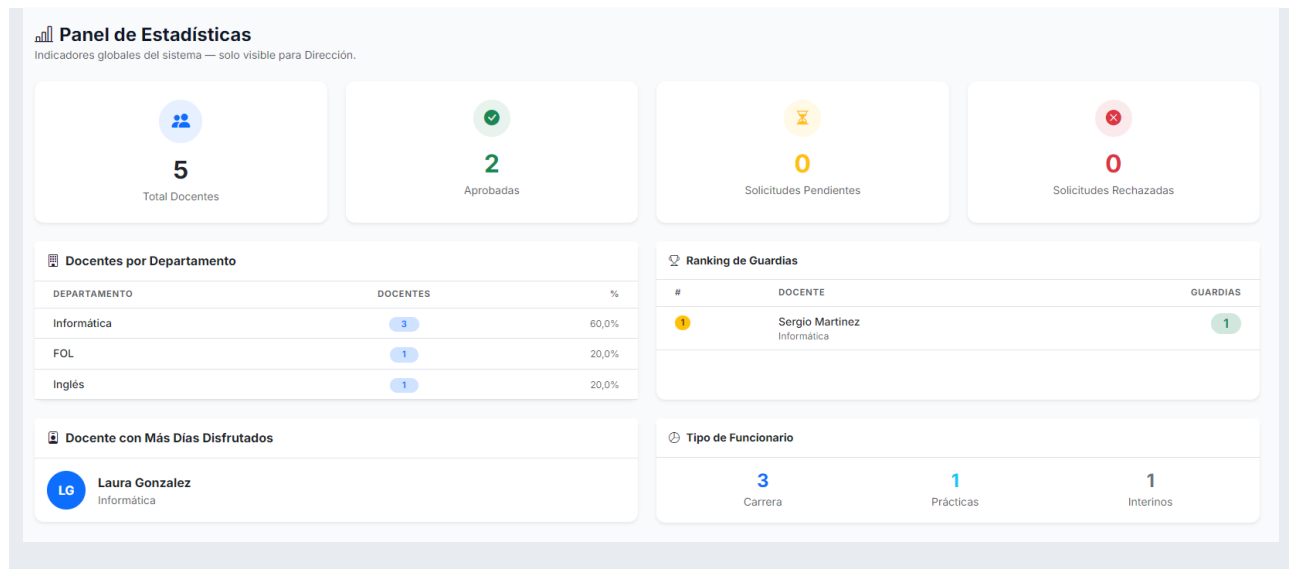


Figura 14. Panel de estadísticas globales (solo Dirección)

4.3. Herramientas de desarrollo

Entornos de desarrollo

Herramienta	Versión	Uso
IntelliJ IDEA Ultimate	2024.1	IDE principal para desarrollo Java/Spring Boot
Visual Studio Code	1.90+	Edición de plantillas Thymeleaf, CSS y configuración
MySQL Workbench	8.0	Administración visual de la base de datos y diagramas ER
Postman	11.x	Pruebas manuales de la API REST
Google Chrome DevTools	—	Depuración de frontend, responsividad, rendimiento
Maven Wrapper (mvnw)	3.9.x	Gestión de dependencias y ciclo de vida del proyecto
JDK	21 (LTS)	Compilación y ejecución de la aplicación

Tabla 23. Entornos y herramientas de desarrollo

Control de versiones

El código fuente del proyecto se gestiona mediante Git como sistema de control de versiones distribuido, y se aloja en un repositorio remoto en GitHub. El flujo de trabajo adoptado sigue un modelo simplificado de ramas:

- Rama main: contiene el código estable y desplegado en producción.
- Ramas de feature: se crean para desarrollar funcionalidades específicas y se fusionan en main mediante pull requests una vez completadas y verificadas.
- Commits semánticos: los mensajes de commit siguen un formato descriptivo que facilita la trazabilidad del historial de cambios.

4.4. Despliegue

Contenerización con Docker

Para el despliegue de la aplicación se ha optado por una estrategia de contenerización con Docker, lo que garantiza portabilidad, reproducibilidad y aislamiento del entorno. La aplicación se empaqueta en una imagen Docker que contiene el JAR compilado sobre un JRE ligero, y se orquesta junto con la base de datos mediante Docker Compose.

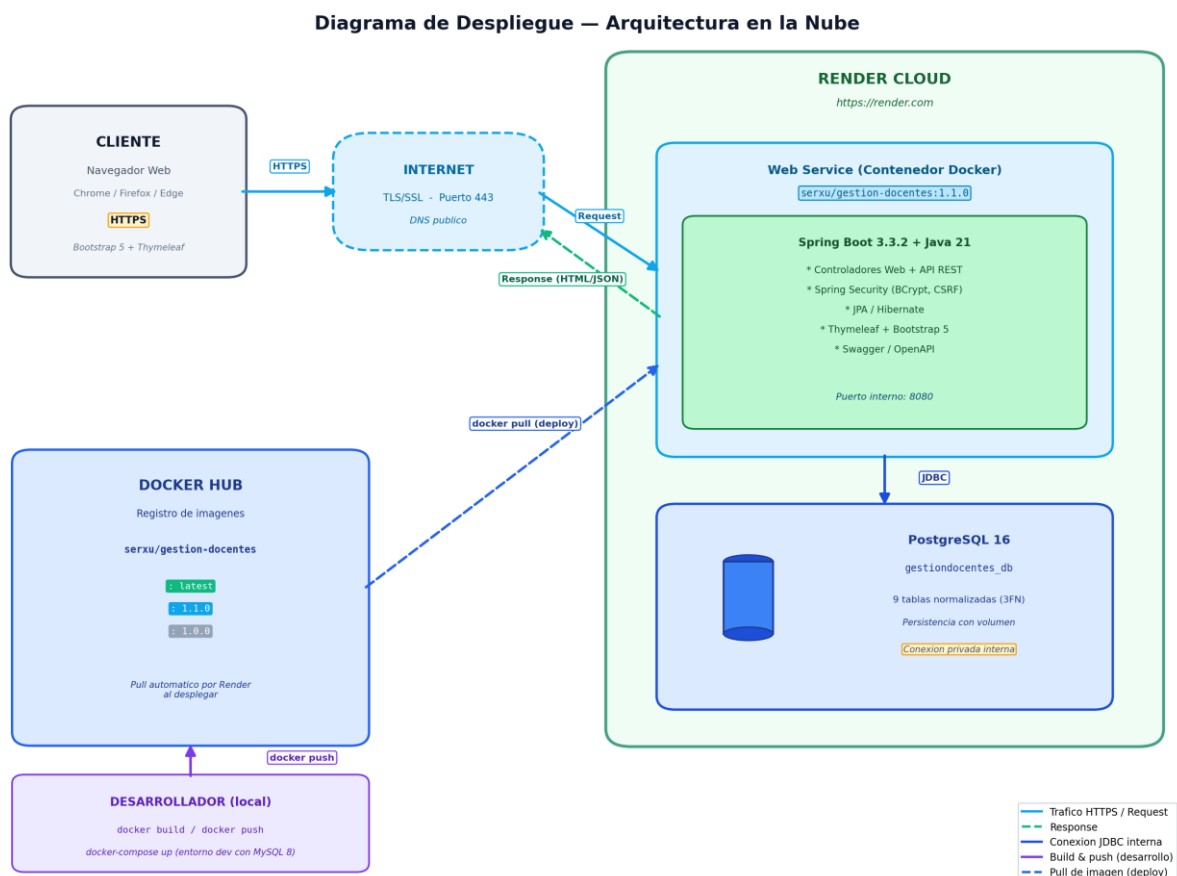


Figura 16. Diagrama de despliegue: arquitectura en la nube con Render y Docker Hub.

Dockerfile (Multi-stage Build)

Se utiliza un Dockerfile multi-stage para minimizar el tamaño de la imagen final y reducir el número de capas generadas. La primera etapa (build) utiliza una imagen con JDK 21 y Maven

para compilar el proyecto. La segunda etapa (runtime) copia únicamente el JAR resultante sobre una imagen con JRE 21, lo que reduce el tamaño de la imagen final significativamente.

Optimizaciones aplicadas en el Dockerfile:

- Multi-stage build: la imagen final solo contiene el JRE y el JAR, sin JDK ni Maven.
- Caché de dependencias: se copian pom.xml y el wrapper de Maven antes del código fuente. Si las dependencias no cambian, Docker reutiliza la capa cacheada, acelerando las reconstrucciones.
- Usuario no-root: el contenedor se ejecuta como un usuario sin privilegios (appuser) por seguridad.
- Health check: verificación automática de que la aplicación responde correctamente en el endpoint /login.
- Variables de entorno: JAVA_OPTS y PORT son configurables sin necesidad de reconstruir la imagen.

Docker Compose (compose.yaml)

El fichero compose.yaml orquesta los dos servicios necesarios para el funcionamiento completo de la aplicación: el contenedor de la aplicación Spring Boot y el contenedor de la base de datos MySQL 8.0. La comunicación entre ambos contenedores se realiza a través de una red Docker interna (gestiondocentes-network), y la aplicación espera a que MySQL esté completamente operativo (healthcheck) antes de arrancar.

Persistencia de datos

La persistencia se garantiza mediante volúmenes nombrados de Docker. El volumen mysql_data almacena los ficheros de datos de MySQL (/var/lib/mysql), asegurando que la información sobrevive a reinicios y recreaciones de contenedores. El volumen uploads_data persiste los archivos subidos por los docentes como material justificativo.

Publicación en Docker Hub

La imagen Docker se ha publicado en Docker Hub de forma pública, con nombre descriptivo y versionado mediante tags. El repositorio es accesible en: <https://hub.docker.com/r/serxu/gestion-docentes>. Las etiquetas publicadas son: latest (versión

estable más reciente) y 1.0.0 (primera versión de producción). El repositorio incluye documentación con instrucciones de uso, variables de entorno disponibles y ejemplo de Docker Compose.

Despliegue en Render

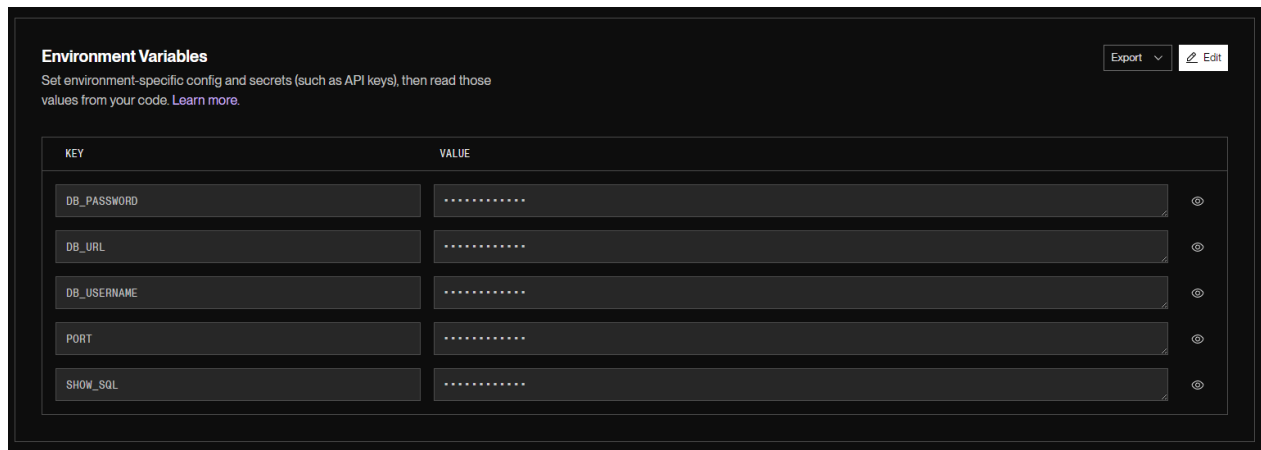
La aplicación se ha desplegado en Render (<https://render.com>), un servicio de hosting en la nube que permite desplegar contenedores Docker de forma gratuita. Se han creado dos recursos en Render:

- Base de datos PostgreSQL: Render no ofrece MySQL de forma nativa, por lo que se utiliza PostgreSQL como base de datos en producción. Gracias a JPA/Hibernate como capa de abstracción, la aplicación es compatible con ambos motores sin cambios en el código, únicamente modificando la URL JDBC de conexión. Para ello se añadió el driver de PostgreSQL al pom.xml como dependencia runtime.
- Web Service: despliega la imagen Docker directamente desde Docker Hub ([docker.io/serxu/gestion-docentes:latest](https://hub.docker.io/serxu/gestion-docentes:latest)). Las variables de entorno (DB_URL, DB_USERNAME, DB_PASSWORD, PORT) se configuran en el panel de Render para conectar con la base de datos PostgreSQL.

La aplicación está accesible públicamente en: <https://gestion-docentes-latest.onrender.com>

Variables de entorno del despliegue

La configuración de la aplicación se ha externalizado completamente mediante variables de entorno, lo que permite adaptar el comportamiento de la aplicación a diferentes entornos (desarrollo local, Docker Compose, Render) sin modificar el código fuente. Las principales variables son: DB_URL (URL JDBC de conexión a la base de datos), DB_USERNAME y DB_PASSWORD (credenciales de la base de datos), PORT (puerto del servidor, por defecto 8080), SHOW_SQL (activar/desactivar el log de consultas SQL) y JAVA_OPTS (opciones de la JVM como memoria máxima).



Usuarios de prueba

Al iniciar la aplicación por primera vez con la base de datos vacía, el sistema crea automáticamente los roles (Dirección, Jefatura, Profesor), los departamentos y un conjunto de usuarios de prueba cargados desde un fichero CSV. Todos los usuarios tienen la contraseña temporal 1234, y el sistema obliga a cambiarla en el primer inicio de sesión (excepto el administrador). A continuación se detallan las credenciales de acceso para realizar pruebas:

Administrador (rol Dirección, acceso completo): Email: `admin@educastur.org` | Contraseña: 1234. Este usuario no necesita cambiar la contraseña y tiene acceso a todas las funcionalidades: gestión de docentes, validación de solicitudes, asignación de guardias, estadísticas, gestión de horarios y API Swagger.

María Suárez (rol Jefatura): Email: `maria@educastur.org` | Contraseña: 1234. Pertenece al departamento de FOL. Puede validar solicitudes, crear y editar docentes, y gestionar horarios.

Sergio Martínez (rol Profesor): Email: `sergio@educastur.org` | Contraseña: 1234. Pertenece al departamento de Informática. Puede consultar su horario, solicitar permisos y ver el historial de solicitudes.

Laura González (rol Profesor): Email: `laura@educastur.org` | Contraseña: 1234. Pertenece al departamento de Informática.

David Álvarez (rol Profesor): Email: `david@educastur.org` | Contraseña: 1234. Pertenece al departamento de Inglés.

Nota: Los usuarios con rol Profesor deberán cambiar la contraseña temporal en su primer acceso. El administrador (`admin@educastur.org`) mantiene su contraseña fija en cada reinicio de la aplicación para facilitar el acceso durante las pruebas.

Medidas de seguridad implementadas

Se han aplicado las siguientes medidas de seguridad para proteger la aplicación:

- API REST autenticada: todos los endpoints bajo `/api/**` requieren sesión HTTP activa. La protección CSRF se desactiva solo para la API para permitir el consumo desde clientes REST externos.
- Validación de archivos subidos: se implementa una lista blanca (whitelist) de extensiones permitidas (.pdf, .doc, .docx, .xls, .xlsx, .jpg, .png, etc.) para prevenir la subida de archivos ejecutables maliciosos.
- Nombres de archivo sanitizados: los archivos subidos se renombran con un patrón seguro (solicitud_ID_timestamp.ext) para prevenir ataques de path traversal.
- Logging profesional con SLF4J: se sustituyen todas las llamadas System.out/System.err por el framework de logging SLF4J, siguiendo las mejores prácticas de la industria.
- Usuario no-root en Docker: el contenedor se ejecuta con un usuario sin privilegios (appuser) para minimizar la superficie de ataque.
- Health check automático: el contenedor Docker verifica periódicamente que la aplicación responde correctamente.
- Cifrado BCrypt: todas las contraseñas se almacenan cifradas con BCrypt (10 rondas).
- Protección CSRF: activa en todos los formularios web.
- Cambio obligatorio de contraseña temporal: el sistema redirige automáticamente al formulario de cambio en el primer login.

5. PLANIFICACIÓN Y PRESUPUESTO

5.1. Diagrama de Gantt

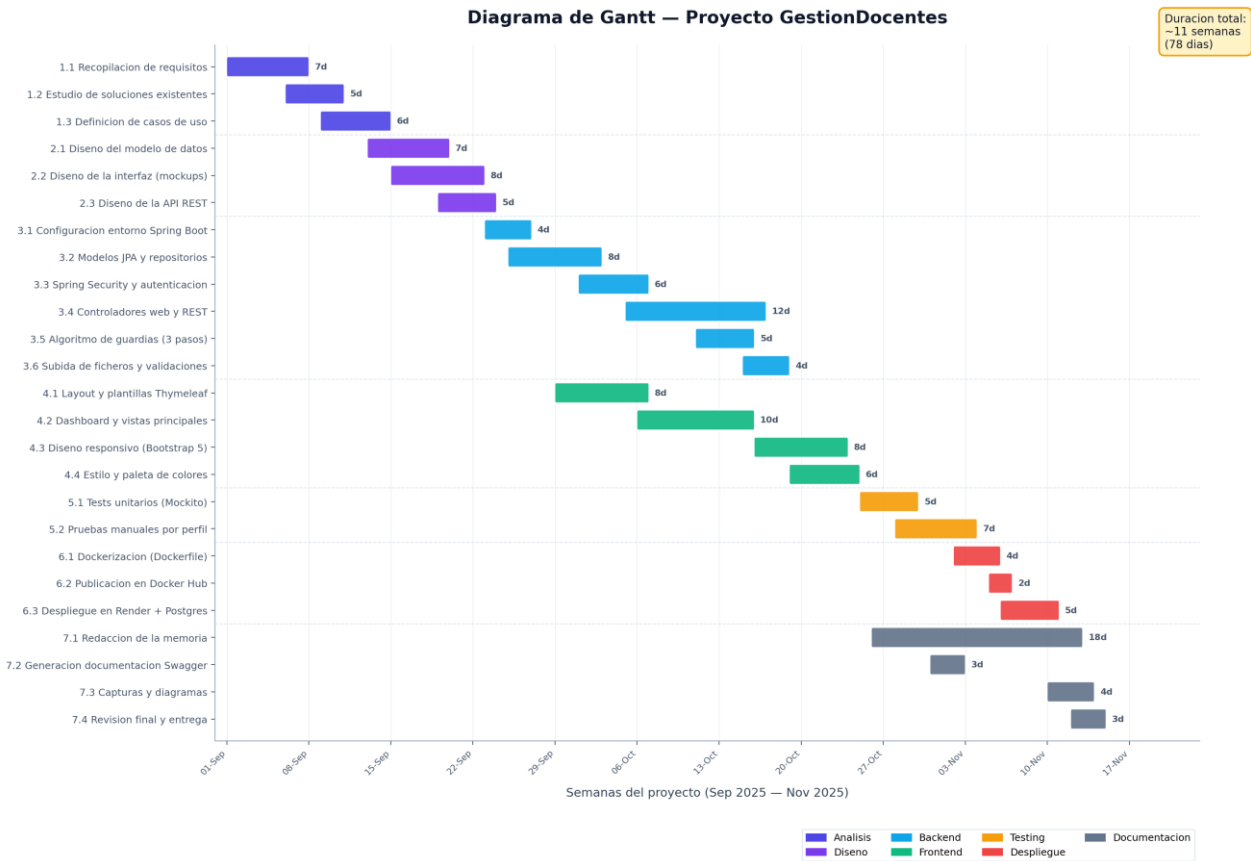


Figura 17. Diagrama de Gantt: planificación temporal del proyecto.

La planificación del proyecto se ha estructurado en 6 fases principales, desarrolladas a lo largo de 14 semanas (desde febrero hasta mayo de 2026). La siguiente tabla representa la distribución temporal de las tareas:

Fase	Tarea	Inicio	Fin	Duración
1. Análisis	Estudio de requisitos y normativa	03/02/2026	14/02/2026	2 semanas
1. Análisis	Diseño del modelo de datos	10/02/2026	21/02/2026	2 semanas
2. Diseño	Diseño de mockups	17/02/2026	28/02/2026	2 semanas

	y guía de estilos			
2. Diseño	Definición de la arquitectura	24/02/2026	07/03/2026	1,5 semanas
3. Backend	Modelo de datos y repositorios	03/03/2026	14/03/2026	2 semanas
3. Backend	Lógica de negocio (servicios)	10/03/2026	21/03/2026	2 semanas
3. Backend	Seguridad y autenticación	17/03/2026	28/03/2026	2 semanas
3. Backend	API REST y documentación Swagger	24/03/2026	04/04/2026	1,5 semanas
4. Frontend	Layout y componentes comunes	31/03/2026	11/04/2026	2 semanas
4. Frontend	Vistas CRUD de docentes	07/04/2026	18/04/2026	2 semanas
4. Frontend	Módulo de solicitudes y guardias	14/04/2026	25/04/2026	2 semanas
4. Frontend	Dashboard diferenciado por roles	21/04/2026	02/05/2026	1,5 semanas
5. Despliegue	Configuración del servidor	28/04/2026	05/05/2026	1 semana
5. Despliegue	Despliegue y pruebas en producción	05/05/2026	09/05/2026	1 semana
6. Documentación	Redacción de la memoria	28/04/2026	13/05/2026	2 semanas
6. Documentación	Grabación del vídeo de exposición	10/05/2026	13/05/2026	3 días

Tabla 25. Planificación temporal del proyecto (Diagrama de Gantt)

5.2. Presupuesto

El presupuesto del proyecto se desglosa en costes de recursos humanos, infraestructura tecnológica y licencias de software. Dado que se trata de un proyecto académico, se han utilizado herramientas gratuitas o con licencia educativa siempre que ha sido posible.

Recursos humanos:

Rol	Horas estimadas	Coste/hora	Total
Analista/Diseñador	40 horas	35 €/h	1.400 €
Desarrollador Backend	120 horas	40 €/h	4.800 €
Desarrollador Frontend	80 horas	35 €/h	2.800 €
Administrador de sistemas	20 horas	30 €/h	600 €
Documentalista	30 horas	25 €/h	750 €
	TOTAL RRHH		10.350 €

Tabla 26. Presupuesto de recursos humanos

Infraestructura y licencias:

Concepto	Coste mensual	Meses	Total
Servidor VPS (2 GB RAM, 40 GB SSD)	10 €/mes	4	40 €
Dominio .es	—	—	10 €/año
Certificado SSL (Let's Encrypt)	Gratuito	—	0 €
IntelliJ IDEA (licencia educativa)	Gratuito	—	0 €
GitHub (plan gratuito)	Gratuito	—	0 €
MySQL Community Edition	Gratuito	—	0 €
		TOTAL INFRA	50 €

Tabla 27. Presupuesto de infraestructura y licencias

Presupuesto total estimado: 10.400 €

6. FUTURAS MEJORAS Y AMPLIACIONES

A continuación se enumeran las mejoras y ampliaciones identificadas durante el proceso de desarrollo que, por limitaciones temporales del proyecto académico, no han sido implementadas en la versión actual pero que resultan viables y deseables para futuras iteraciones:

Aplicación móvil nativa: Desarrollo de una aplicación para Android e iOS que consuma la API REST existente, permitiendo a los docentes consultar su horario, solicitar permisos y recibir notificaciones push desde el dispositivo móvil.

Notificaciones en tiempo real: Implementación de WebSockets (Spring WebSocket / STOMP) para notificar al usuario en tiempo real cuando su solicitud sea aprobada o rechazada, sin necesidad de recargar la página.

Módulo de evaluaciones: Ampliación del sistema con un módulo de seguimiento del rendimiento académico del alumnado, integrado con las asignaturas y los horarios existentes.

Generación automática de horarios: Incorporación de un algoritmo de optimización combinatoria para la generación automática de cuadrantes horarios, respetando restricciones de disponibilidad y aulas.

Autenticación con LDAP/Active Directory: Integración con el directorio de usuarios del centro educativo (Educastur) para centralizar la gestión de credenciales.

Panel de informes avanzados: Generación de informes en formato PDF con gráficos estadísticos (Chart.js o JasperReports) sobre el uso de permisos y distribución de guardias.

Internacionalización (i18n): Soporte multiidioma (castellano, asturiano, inglés) mediante los mecanismos de internacionalización de Spring Boot y Thymeleaf.

Tests automatizados: Desarrollo de una batería de tests unitarios (JUnit 5, Mockito) y tests de integración (Spring Boot Test, Testcontainers) para garantizar la no regresión.

Pipeline CI/CD: Configuración de un pipeline de integración y despliegue continuo con GitHub Actions que ejecute tests, compile el proyecto y despliegue automáticamente.

Gestión de aulas y espacios: Módulo para la gestión del inventario de aulas, laboratorios y talleres, con control de ocupación y reservas vinculadas a los horarios.

7. VALORACIÓN Y CONCLUSIONES

Dificultades técnicas encontradas

Durante el desarrollo de GestiónDocentes se presentaron diversas dificultades técnicas que requirieron investigación, experimentación y, en ocasiones, cambios de enfoque. A continuación se detallan las más significativas:

Serialización JSON con relaciones bidireccionales: Las relaciones bidireccionales entre entidades JPA (por ejemplo, Docente y AsuntoPropio) provocaban recursión infinita al serializar los objetos a JSON para la API REST. La solución consistió en utilizar combinaciones de `@JsonIgnore`, `@JsonIgnoreProperties` y `@JsonProperty(access = WRITE_ONLY)` para controlar la profundidad de serialización sin perder información relevante.

Configuración de seguridad con múltiples roles: La configuración de Spring Security para tres roles con permisos superpuestos generó conflictos iniciales. Por ejemplo, las rutas `/web/admin/**` estaban restringidas a ADMIN, lo que bloqueaba el acceso de los Jefes de Departamento al panel de validación. La resolución requirió segmentar las reglas de acceso: `/web/admin/estadisticas` exclusivamente para ADMIN y `/web/admin/**` para ambos roles.

Eliminación en cascada con integridad referencial: La eliminación de un docente con registros asociados en múltiples tablas (solicitudes, horarios, faltas, tokens, referencias como sustituto) provocaba excepciones de restricción de clave foránea. Se resolvió implementando una eliminación transaccional (`@Transactional`) que limpia ordenadamente todas las dependencias antes de eliminar el registro principal.

Validación de reglas de negocio complejas: Las reglas de negocio para las solicitudes de permisos (fecha no pasada, cupo diario, límite trimestral) requerían consultas específicas y lógica condicional que no podía resolverse únicamente con Bean Validation. Se optó por una validación programática en el controlador que complementa las anotaciones declarativas del modelo.

Integración del servidor SMTP en entorno de desarrollo: El envío de correos electrónicos requiere un servidor SMTP funcional que no siempre está disponible en entornos de desarrollo local. Se implementó un bloque try-catch que registra el enlace de recuperación en la consola cuando el envío falla, permitiendo continuar el desarrollo sin depender de un servidor de correo externo.

Valoración personal del aprendizaje intermodular

El desarrollo de este proyecto intermodular ha supuesto una experiencia formativa de gran valor, ya que ha permitido integrar en una única aplicación funcional los conocimientos adquiridos en los cuatro módulos del segundo curso del ciclo de Desarrollo de Aplicaciones Web.

Desarrollo Web en Entorno Servidor (DWES):

Este módulo constituye el núcleo tecnológico del proyecto. La implementación del backend con Spring Boot ha permitido profundizar en la arquitectura MVC, la inyección de dependencias, la persistencia con JPA/Hibernate, la seguridad con Spring Security y la creación de API REST. La gestión de transacciones, las consultas JPQL personalizadas y la configuración de múltiples capas de seguridad han sido los aspectos más enriquecedores desde el punto de vista técnico.

Desarrollo Web en Entorno Cliente (DWEC):

Aunque el proyecto utiliza renderizado del lado del servidor con Thymeleaf, se ha empleado JavaScript nativo para las interacciones del lado del cliente: diálogos de confirmación en eliminaciones, validaciones de formularios en tiempo real y la dinamización de ciertos componentes. La integración con el DOM y la gestión de eventos han reforzado los conceptos adquiridos en este módulo.

Diseño de Interfaces Web (DIW):

El diseño visual de la aplicación refleja los principios de usabilidad, accesibilidad y diseño responsivo estudiados en este módulo. La utilización de Bootstrap 5 como framework CSS, la definición de una paleta de colores coherente, la elección tipográfica (Inter), el diseño del sistema de layout con sidebar y la adaptación a dispositivos móviles son resultado directo de las competencias adquiridas en Diseño de Interfaces.

Despliegue de Aplicaciones Web (DAW):

La contenerización del proyecto con Docker (Dockerfile multi-stage, Docker Compose), la publicación de la imagen en Docker Hub y el despliegue en producción en Render con base de datos PostgreSQL en la nube han puesto en práctica los conocimientos de administración de servidores y despliegue estudiados en este módulo. La externalización de la configuración mediante variables de entorno, la compatibilidad dual MySQL/PostgreSQL y el uso de Docker Compose con healthchecks completan el ciclo de despliegue.

Conclusión general:

GestiónDocentes representa una solución funcional, segura y extensible para la gestión integral del profesorado en centros de Formación Profesional. El proyecto demuestra la capacidad de integrar tecnologías heterogéneas en una aplicación cohesiva, desde la base de datos relacional hasta la interfaz de usuario responsiva, pasando por la API REST documentada y el despliegue en producción. La experiencia adquirida durante su desarrollo constituye un fundamento sólido para afrontar proyectos profesionales de mayor envergadura en el futuro.

8. ÍNDICE DE FIGURAS Y TABLAS

Índice de figuras

- Figura 1. Vista general del dashboard de la aplicación (rol Dirección)
- Figura 2. Pantalla de inicio de sesión de la aplicación
- Figura 3. Sidebar de navegación y cabecera con perfil de usuario
- Figura 4. Ejemplo de tarjetas del dashboard con estadísticas
- Figura 5. Diagrama Entidad-Relación de la base de datos
- Figura 6. Documentación interactiva de la API en Swagger UI
- Figura 7. Listado de docentes con paginación, búsqueda y filtros
- Figura 8. Formulario de creación/edición de docente con validación inline
- Figura 9. Perfil detallado de un docente con horario semanal y solicitudes
- Figura 10. Formulario de solicitud de día de asuntos propios
- Figura 11. Panel de validación de solicitudes (vista administrador)
- Figura 12. Calendario de guardias y asignación automática de sustitutos
- Figura 13. Rejilla de horario semanal del docente
- Figura 14. Panel de estadísticas globales (solo Dirección)
- Figura 15. Arquitectura por capas (MVC) del proyecto
- Figura 16. Diagrama de despliegue en la nube (Render + Docker Hub)
- Figura 17. Diagrama de Gantt: planificación temporal del proyecto

Índice de tablas

- Tabla 1. Comparativa de soluciones existentes frente a GestiónDocentes
- Tabla 2. Requisitos no funcionales del sistema
- Tabla 3. Requisitos funcionales del sistema
- Tabla 4. Casos de uso del actor Profesor
- Tabla 5. Casos de uso del actor Jefe de Departamento
- Tabla 6. Casos de uso del actor Dirección
- Tabla 7. Paleta de colores de la interfaz
- Tabla 8. Diccionario de datos — Tabla docente
- Tabla 9. Diccionario de datos — Tabla departamento
- Tabla 10. Diccionario de datos — Tabla rol

Tabla 11. Diccionario de datos — Tabla asunto_propio

Tabla 12. Diccionario de datos — Tabla horario

Tabla 13. Diccionario de datos — Tabla asignatura

Tabla 14. Diccionario de datos — Tabla ciclo

Tabla 15. Diccionario de datos — Tabla falta

Tabla 16. Diccionario de datos — Tabla password_reset_token

Tabla 17. Endpoints de la API — Recurso Docentes

Tabla 18. Endpoints de la API — Recurso Solicitudes

Tabla 19. Endpoints de la API — Recurso Departamentos

Tabla 20. Endpoints de la API — Recurso Horarios

Tabla 21. Dependencias principales del proyecto

Tabla 22. Tecnologías del frontend

Tabla 23. Entornos y herramientas de desarrollo

Tabla 24. Especificaciones del entorno de despliegue

Tabla 25. Planificación temporal del proyecto (Diagrama de Gantt)

Tabla 26. Presupuesto de recursos humanos

Tabla 27. Presupuesto de infraestructura y licencias

9. BIBLIOGRAFÍA/WEBGRAFÍA

Las siguientes referencias bibliográficas y webgráficas han sido consultadas durante el desarrollo del proyecto. Se presentan ordenadas alfabéticamente según el estilo APA (7.ª edición).

Baeldung. (2024). *Spring Boot Tutorial*. <https://www.baeldung.com/spring-boot>

Bootstrap. (2023). *Bootstrap 5.3 Documentation*. <https://getbootstrap.com/docs/5.3/>

Bootstrap Icons. (2023). *Bootstrap Icons Library*. <https://icons.getbootstrap.com/>

Google Fonts. (2024). *Inter Font Family*. <https://fonts.google.com/specimen/Inter>

Hibernate. (2024). *Hibernate ORM Documentation*. <https://hibernate.org/orm/documentation/>

Mozilla Developer Network. (2024). *Referencia de JavaScript*.
<https://developer.mozilla.org/es/docs/Web/JavaScript>

MySQL. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

Oracle. (2024). *Java SE 21 Documentation*. <https://docs.oracle.com/en/java/javase/21/>

Pivotal Software. (2024a). *Spring Boot Reference Documentation (v3.3.2)*. <https://docs.spring.io/spring-boot/docs/3.3.2/reference/html/>

Pivotal Software. (2024b). *Spring Data JPA Reference Documentation*. <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>

Pivotal Software. (2024c). *Spring Security Reference Documentation*. <https://docs.spring.io/spring-security/reference/>

Springdoc. (2024). *Springdoc OpenAPI Documentation*. <https://springdoc.org/>

Thymeleaf. (2024). *Thymeleaf Documentation*. <https://www.thymeleaf.org/documentation.html>

W3C. (2024). *Web Content Accessibility Guidelines (WCAG) 2.1*. <https://www.w3.org/TR/WCAG21/>

ANEXOS

Anexo A. Capturas de pantalla de la aplicación

Las capturas de pantalla de la aplicación en funcionamiento se incluyen en el vídeo de exposición que acompaña a esta memoria. En dicho vídeo se muestra la aplicación desplegada en el entorno de producción, con la URL visible en todo momento en la barra de direcciones del navegador. Se recorren las siguientes funcionalidades:

- Pantalla de login y proceso de autenticación.
- Dashboard diferenciado para cada rol (Dirección, Jefe de Departamento, Profesor).
- Listado de docentes con paginación, búsqueda y filtros por departamento.
- Formulario de creación de docente con validación inline.
- Formulario de edición de docente.
- Perfil detallado de un docente con horario y solicitudes.
- Exportación del claustro en formato CSV.
- Formulario de solicitud de día de asuntos propios.
- Historial personal de solicitudes con estadísticas.
- Panel de validación de solicitudes (vista administrador).
- Calendario de guardias y asignación automática de sustitutos.
- Rejilla de horario semanal del docente.
- Formulario de gestión de horarios (asignación de franjas).
- Panel de estadísticas globales (solo Dirección).
- Documentación interactiva de la API (Swagger UI).
- Páginas de error personalizadas (404, 500, acceso denegado).

Anexo B. Estructura del proyecto

A continuación, se muestra la estructura de directorios del proyecto, siguiendo la convención estándar de un proyecto Maven con Spring Boot:

```
gestiondocentes/  
src/  
main/  
java/com/sergiovd/gestiondocentes/  
config/
```

```
AdminLoader.java
CsvDataLoader.java
GlobalExceptionHandler.java
OpenApiConfig.java
SecurityConfig.java
controller/
  AuthController.java
  DocenteController.java
  GlobalController.java
  CustomErrorController.java
api/
  DepartamentoApiController.java
  DocenteApiController.java
  HorarioApiController.java
  SolicitudApiController.java
dto/
  SolicitudDTO.java
model/
  Asignatura.java, AsuntoPropio.java, Ciclo.java,
  Departamento.java, Docente.java, Faltas.java,
  Horario.java, PasswordResetToken.java, Rol.java
repository/
  (9 interfaces JpaRepository)
service/
  DocenteService.java, DocenteServiceImpl.java,
  GuardiaService.java, UserDetailsServiceImpl.java
resources/
  templates/ (20 plantillas Thymeleaf)
  application.properties
  docentes.csv
pom.xml
mvnw / mvnw.cmd
```